

```
// =====
// clinical-logic-compat.ts
// -----
// Camada de COMPATIBILIDADE com o clinical-logic.ts original.
// Mantém a API pública idêntica enquanto internamente usa o motor refatorado.
//
// ESTRATÉGIA DE MIGRAÇÃO:
// 1. Renomear clinical-logic.ts → clinical-logic.original.ts (backup)
// 2. Este arquivo vira o novo clinical-logic.ts
// 3. Imports no Dashboard.tsx e PatientCard.tsx NÃO precisam mudar
// 4. Gradualmente substituir as importações diretas pelas novas
//
// O que mudou INTERNAMENTE (transparente pro componente):
//   Bug #1: SOFA Cardio usa dose real
//   Bug #2: SOFA Neuro suprimido sob sedação
//   Bug #3: Sepsis-3 com ΔSOFA (fallback para score absoluto enquanto sem
timeseries)
//   Bug #4: Critérios de infecção estruturados
//   Bug #5: SOFA Resp diferencia VM/não-VM
//   Bug #6: SOFA Renal inclui DU e TRRC
//   Bug #7: parseFloatBR em todos os cálculos
//   Bug #8: coercePlaquetas com unidade
//   Bug #9: coerceFiO2Input em P/F
//   Bug #11: pam1 é MIN (pior valor)
//   Bug #12: ID via crypto.randomUUID
// =====

import type { Patient } from './types';
import { getSOFA } from './scores/sofa';
import { assessSepsis } from './scores/sepsis';
import { getQSOFA } from './scores/qsofa';
import { calcDoseInfusao } from './calculations/infusao';
import { calcDiureseEfetiva } from './calculations/diurese';
import { parseFloatBR } from './calculations/parseBR';
import { DVA_DICT, SEDACAO_DICT, ESCALAS_NEURO_DICT } from './dictionaries';

export { DVA_DICT, SEDACAO_DICT, ESCALAS_NEURO_DICT };

// =====
// getInitialState – Bug #12 fix: crypto.randomUUID em vez de Math.random
// =====
export const getInitialState = (uti = 'UTI 2', id: string | null = null): Patient =>
({
  id: id ?? (
    typeof crypto !== 'undefined' && crypto.randomUUID()
    ? crypto.randomUUID()
    : Math.random().toString(36).substring(2, 9) // fallback SSR
  ),
  uti: uti as any,
  nome: '',
  leito: '',
  hd: '',
  adm: new Date().toLocaleDateString('pt-BR'),
  peso: '',
  altura: '',
  alergias: '',
  gravidade: '',
  dvas: [],
  sedativos: [],
  neuro: {
    escalas: [],
    pupilas: 'Isototoreagentes, sem déficits focais',
    analgesia: '',
    camIcu: '',
    notas: '',
  },
},

```

```
resp: {
  suporte: '',
  dataIntubacao: '',
  vazaoO2: '',
  fio2O2: '',
  vmModo: '',
  vmPeep: '',
  vmFio2: '',
  vmVc: '',
  vmPinsps: '',
  pao2: '',
  fr1: '',
  fr2: '',
  frX: '',
  spo2: '',
  spo2X: '',
  ausculta: '',
  notas: '',
},
hemo: {
  pas1: '',
  pas2: '',
  pasX180: '',
  pasX100: '',
  pad1: '',
  pad2: '',
  padX120: '',
  padX50: '',
  pam1: '',
  pam2: '',
  pamX130: '',
  pamX65: '',
  fc1: '',
  fc2: '',
  fcX100: '',
  lactato: '',
  ausculta: 'BNF RR 2T SS.',
  pele: 'TEC < 3s | Extremidades quentes, bem perfundidas. MMII s/ edema s/ TVP.',
  notas: '',
},
tgi: {
  dx: '',
  dxX180: '',
  abdome: 'Semi-globoso, flácido, RHA +, sem sinais de peritonite.',
  bb: '',
  viaDieta: '',
  vazaoDieta: '',
  dietaOutra: '',
  aceitacao: '',
  evacuou: '',
  evacuacoesNum: '',
  evacuacoesAspecto: '',
  evacuacoesDataUltima: '',
  kcalMeta: '',
  kcalInfundido: '',
  notas: '',
},
renal: {
  ur1: '',
  ur2: '',
  ur3: '',
  cr1: '',
  cr2: '',
  cr3: '',
  tipoDiurese: '',
  diurese: '',
}
```

```

    diureseHoras: '24',
    bh: '',
    mg: '',
    na: '',
    cai: '',
    k: '',
    trrc: false,
    notas: '',
  },
  hemato: {
    hb1: '',
    ht1: '',
    plaq1: '',
    plaqUnit: '×103/μL',
    profilaxiaTvp: '',
    profilaxiaUlcera: '',
    notas: '',
  },
  infecto: {
    tmax: '',
    tmaxX38: '',
    tmin: '',
    atbs: [],
    culturas: [],
    leuco1: '',
    leuco2: '',
    leuco3: '',
    focoSuspeito: '',
    notas: '',
  },
  impressao: ['', '', '', ''],
  conduta: ['', '', '', ''],
  pendencias: [
    { checked: false, text: '' },
    { checked: false, text: '' },
    { checked: false, text: '' },
  ],
});

// =====
// getClinicalIntelligence – mantém API pública mas usa motor corrigido
// =====
export const getClinicalIntelligence = (p: Patient) => {
  const sofaResult = getSOFA(p);
  const sepsisResult = assessSepsis(
    sofaResult,
    null, // sem baseline ainda (timeseries pendente)
    p.infecto,
    p.hemo,
    p.dvas,
  );
  const qsofaResult = getQSOFA(p);

  return {
    sofa: sofaResult.total,
    sofaDet: sofaResult.detail,
    sofaComps: sofaResult.components,
    missing: sofaResult.missing,
    suppressed: sofaResult.suppressed,
    warnings: sofaResult.warnings,
    isSeptic: sepsisResult.isSeptic,
    isSepticShock: sepsisResult.isSepticShock,
    sepsisReason: sepsisResult.reason,
    sepsisEvidences: sepsisResult.infectionEvidences,
    qsofa: qsofaResult,
    // Campos extras novos (UI pode optar por mostrar)
  };
};

```

```

    lactato: sepsisResult.lactato,
    sepsisWarnings: sepsisResult.warnings,
  };
};

// =====
// calcDoseInfusao - re-exporta com assinatura compatível
// =====
export const calcDoseInfusaoCompat = (
  infusao: any,
  dictMap: any,
  pesoAtual: any,
) => {
  const dict = dictMap?.[infusao?.droga];
  if (!dict) return null;

  const result = calcDoseInfusao(infusao, dict, pesoAtual);
  if (!result || result.error) {
    if (result?.error === 'missing_weight') return { error: 'Insira o Peso' };
    return null;
  }

  return {
    value: result.value?.toFixed(2) ?? '0.00',
    unit: result.unit,
    min: result.min,
    max: result.max,
    isOk: result.isInRange,
  };
};

// =====
// calcDiureseEfetiva - re-exporta com assinatura compatível
// =====
export const calcDiureseEfetivaCompat = (
  diurese: any,
  peso: any,
  horas: any,
): string => {
  const result = calcDiureseEfetiva(diurese, peso, horas);
  return result.formatted;
};

// =====
// isHigh / isLow - Bug #7 fix: parseFloatBR em vez de parseFloat
// =====
export const isHigh = (v1: any, v2: any, threshold: number): boolean => {
  const n1 = parseFloatBR(v1);
  const n2 = parseFloatBR(v2);
  return (!isNaN(n1) && n1 > threshold) || (!isNaN(n2) && n2 > threshold);
};

export const isLow = (v1: any, v2: any, threshold: number): boolean => {
  const n1 = parseFloatBR(v1);
  const n2 = parseFloatBR(v2);
  return (!isNaN(n1) && v1 !== '' && n1 < threshold) ||
    (!isNaN(n2) && v2 !== '' && n2 < threshold);
};

// =====
// generateSinglePatientText - mantido para "copy all patients"
// =====
export const generateSinglePatientText = (d: Patient): string => {
  const intell = getClinicalIntelligence(d);

  const getCamIcu = () =>

```

```

d.neuro.camIcu === 'Positivo' ? '[X] Positivo | [ ] Negativo' :
d.neuro.camIcu === 'Negativo' ? '[ ] Positivo | [X] Negativo' :
                                '[ ] Positivo | [ ] Negativo';

const getAuscultaResp = (tipo: string) => d.resp.ausculta === tipo ? '[X]' : '[ ]';

const pendenciasText = d.pendencias
  .map(p => `[${p.checked ? 'X' : ' '}] ${p.text} ||
  _____ '}`)
  .join('\n');

const profs = [d.hemato.profilaxiaTvp, d.hemato.profilaxiaUlcera]
  .filter(Boolean).join(' + ');

const formatInfusaoText = (infusoes: any[], dictMap: any) =>
  infusoes.length > 0
    ? infusoes.map(inf => {
        if (!inf.droga) return '';
        const doseData = calcDoseInfusaoCompat(inf, dictMap, d.peso);
        return `${inf.droga} - Vazão: ${inf.vazao || '___'} ml/h${doseData &&
!doseData.error ? ` -> [ ${doseData.value} ${doseData.unit} ]` : ''}`;
      }).filter(Boolean).join('\n')
    : 'Nenhuma';

const atbsText = d.infecto.atbs.length > 0
  ? d.infecto.atbs.map(a =>
    `${a.nome === 'Outro' ? a.nomePersonalizado : a.nome || '___'} ${a.dose ?
`(${a.dose})` : ''} ${a.dias ? ` - D${a.dias}` : ''}${a.intencao ? ` [${a.intencao}]` :
''}`
  ).filter(Boolean).join('\n')
  : 'Nenhum';

const culturasText = d.infecto.culturas
  .map(c =>
    `${c.tipo || '___'} (${c.data || '___/___'}): ${c.status || '___'}${
(c.status === 'Parcial positiva' || c.status === 'Positiva') && c.detalhe
? ` -> ${c.detalhe}`
: ''
  }`
  ).join('\n');

const printNotas = (notas: string) => notas ? `\n ↳ Notas: ${notas}` : '';

let suporteRespText = d.resp.suporte || 'Ar ambiente';
if (d.resp.suporte === 'IOT + VM') {
  suporteRespText += `(Data IOT: ${d.resp.dataIntubacao || '___'}) -> Modo:
${d.resp.vmModo || '___'} | PEEP: ${d.resp.vmPeep || '___'} | FiO2: ${d.resp.vmFio2 ||
'___'}% | VC: ${d.resp.vmVc || '___'} | P. Insp/PS: ${d.resp.vmPinsps || '___'} |
PaO2: ${d.resp.pao2 || '___'}`;
} else if (d.resp.suporte === 'CNL O2') {
  suporteRespText += `(${d.resp.vazao02 || '___'} L/min) | PaO2: ${d.resp.pao2 ||
'___'}`;
} else if (d.resp.suporte === 'CNAF') {
  suporteRespText += `(Vazão: ${d.resp.vazao02 || '___'} L/min | FiO2:
${d.resp.fio202 || '___'}%) | PaO2: ${d.resp.pao2 || '___'}`;
}

const diurEfet = calcDiureseEfetivaCompat(d.renal.diurese, d.peso,
d.renal.diureseHoras);

return `# PASSAGEM DE TURNO - ${d.uti || 'UTI 2'}
## Leito: ${d.leito || '-'}
Nome/Iniciais: ${d.nome || '_____'}
HD: ${d.hd || '_____'} | Adm: ${d.adm}
Peso: ${d.peso || '___'} kg | Altura: ${d.altura || '___'} m | Gravidade:
${d.gravidade || 'Não classificada'}`

```

```

Alergias: ${d.alergias || 'Nega'}
> SOFA Score Estimado: ${intell.sofa} ${intell.isSepticShock ? '| □□ CHOQUE SÉPTICO' :
intell.isSeptic ? '| △□ SEPSIS-3 DETECTADA' : ''}

${intell.sepsisReason === 'sofa_absoluto_fallback_sem_baseline' ? '> △□ SOFA basal
não disponível – ΔSOFA não calculado\n' : ''}
Avaliação estruturada por Sistemas:
- Neurológico:
Sedação: ${formatInfusaoText(d.sedativos, SEDACAO_DICT)}
Escalas: ${d.neuro.escalas.length > 0 ? d.neuro.escalas.map(esc => `${esc.nome}:
${esc.valor || '___'}${esc.sobSedacao ? ' (sob sedação)' : ''}`).join(' | ') :
'Nenhuma'}
Pupilas: ${d.neuro.pupilas}
Analgesia: ${d.neuro.analgesia || '_____'} Adequada
CAM - ICU: ${getCamIcu()}${printNotas(d.neuro.notas)}

- Respiratório:
Suporte: ${suporteRespText}
FR: ${d.resp.fr1 || '___'} - ${d.resp.fr2 || '___'} irpm | SpO2: ${d.resp.spo2 ||
'___'} %
Ausculta pulmonar:
${getAuscultaResp('MV + Bilateralmente, SRA')} MV + Bilateralmente, SRA
${getAuscultaResp('MV + Bilateralmente, creptações bibasais')} MV + Bilateralmente,
creptações bibasais
${getAuscultaResp('MV + Bilateralemtente, reduzido globalmente, sra')} MV + Reduzido
globalmente, SRA
${getAuscultaResp('MV + Bilateralmente, Roncos difusos')} MV + Bilateralmente, Roncos
difusos
${getAuscultaResp('MV + Bilateralmente, com Sibilos')} MV + Bilateralmente,
Sibilos${printNotas(d.resp.notas)}

- Hemodinâmico:
DVA: ${formatInfusaoText(d.dvas, DVA_DICT)}
PAS: ${d.hemo.pas1 || '___'} - ${d.hemo.pas2 || '___'} mmHg | PAD: ${d.hemo.pad1 ||
'___'} - ${d.hemo.pad2 || '___'} mmHg
PAm: ${d.hemo.pam1 || '___'} - ${d.hemo.pam2 || '___'} mmHg | FC: ${d.hemo.fc1 ||
'___'} - ${d.hemo.fc2 || '___'} bpm
Lactato: ${d.hemo.lactato || '___'} mmol/L
Ausculta: ${d.hemo.ausculta}
Pele/Perfusão: ${d.hemo.pele}${printNotas(d.hemo.notas)}

- TGI:
Abdome: ${d.tgi.abdome}
Dieta: ${d.tgi.viaDieta || '___'}${d.tgi.vazaoDieta ? ` (${d.tgi.vazaoDieta} ml/h)` :
''} | Glicemia: ${d.tgi.dx || '___'} mg/dL
Evacuações: ${d.tgi.evacuou || '___'}
BB: ${d.tgi.bb || '___'} mg/dL${d.tgi.kcalMeta ? ` | Meta Cal: ${d.tgi.kcalMeta} /
Infundido: ${d.tgi.kcalInfundido || '___'} kcal` : ''}${printNotas(d.tgi.notas)}

- Renal/Metabólico:
UR: ${d.renal.ur1 || '___'} | Cr: ${d.renal.cr1 || '___'} mg/dL${d.renal.trrc ? ' |
TRRC ATIVA' : ''}
Diurese (${d.renal.tipoDiurese || '___'}, ${d.renal.diureseHoras}h): ${d.renal.diurese
|| '___'} ml => Ef: ${diurEfet} ml/kg/hr
BH: ${d.renal.bh || '___'} ml${d.renal.bhAcumulado ? ` | BH Acumulado:
${d.renal.bhAcumulado} ml` : ''}
Na: ${d.renal.na || '___'} mEq/L | K: ${d.renal.k || '___'} mEq/L | Mg: ${d.renal.mg
|| '___'} | Ca2+: ${d.renal.cai || '___'}${printNotas(d.renal.notas)}

- Hemato:
Hb: ${d.hemato.hb1 || '___'} g/dL | HT: ${d.hemato.ht1 || '___'} % | PlaQ:
${d.hemato.plaq1 || '___'} ${d.hemato.plaqUnit ?? ''}
Profilaxias: ${profs || '_____'}${printNotas(d.hemato.notas)}

- Infecto:
Tax: ${d.infecto.tmax || '___'} °C | Leucócitos: ${d.infecto.leuco1 || '___'} /µL

```

ATB:

 \${atbsText}

Culturas:

 \${culturasText || ' Nenhuma'}\${printNotas(d.infecto.notas)}

Impressão clínica / Problemas ativos

 \${d.impressao.map(i => i || '-----').join('\n')}

Conduta/Plano 12-24 hrs

 \${d.conduta.map(c => c || '-----').join('\n')}

Pendências / Riscos e contingências

 \${pendenciasText}

 `;

};